Session 14 — Study Guide: Finishing Issues, Opening Authentication

This is what we learned in Session 14. Use it to review before you start your Completion Homework, and to refresh your memory later in the course.

Part A — Completing the Issues work

We closed out the real, unfinished parts of our Issues feature: filtering, documentation, and the index behind it.

Status and title filtering

`GET /issues` now supports three independent, combinable query parameters:

TS

```ts
@ApiPropertyOptional({ example: 1 })
@IsOptional()
@Type(() => Number)
@IsInt()
projectId?: number;

@ApiPropertyOptional({ example: 'todo', enum: ['todo', 'in_progress', 'review', 'done'] })
@IsOptional()
@IsIn(['todo', 'in_progress', 'review', 'done'])
status?: string;

@ApiPropertyOptional({ example: 'login' })
@IsOptional()
@IsString()
title?: string;
```

`status` is validated against the fixed set of real statuses (`@IsIn`) — a typo in a query string gets a clean `400`, not a query that silently returns nothing. `title` uses the same case-insensitive partial match (`$regex` + `$options: 'i'`) you've already seen before in this course — known pattern, applied to a new field.

Every guarded write route on `IssuesController` (create/update/delete) now documents its own real success status and its `403` in Swagger — so the documentation actually matches what the API does, not just what it originally did when the route was first built.

The real compound index

Our `Issue` schema declares one compound index:

TS

```ts
IssueSchema.index({ status: 1, projectId: 1 });
```

Read the field order carefully — `status` comes first, `projectId` second. We created this compound index in class, replacing the single-field index that was there before.

In the Completion Homework, you will investigate exactly which query shapes this field order supports — and which it doesn't — and why field order matters for a compound index in general. You will measure this yourself with `explain("executionStats")`, not take it on faith.

The real Projects bug, and its fix

`ProjectsService.create()` used to crash once every project had ever been deleted, because it read `.id` off a result that could be `null`:

TS

```ts
const last = await this.model.findOne().sort({ id: -1 });
const id = (last?.id || 0) + 1;
```

The `?.` (optional chaining) is what fixes the crash — it short-circuits to `undefined` instead of throwing when `last` is `null`, and the fallback then supplies `0`. This is the same real, source-level null-safety discipline you've already used elsewhere in this project, applied here as a real bug fix, not a new concept.

Part B — Opening Authentication

We started a new part of this application: for the first time, there is a real `User` who can register an account.

Why `User` lives in its own module, separate from Auth

You might expect `User`'s own schema and database logic to live inside an `auth/` folder, next to registration. It doesn't — it lives in its own `users/` module instead:

TEXT

```
users/
  schemas/user.schema.ts
  users.service.ts
  users.module.ts

auth/
  dto/register.dto.ts
  auth.service.ts
  auth.controller.ts
  auth.module.ts
```

The reason: `User` is a real, independent part of this application's data — later features (who's a member of a workspace, who's assigned to an issue, who wrote a comment) will all need to reference a `User`, and none of those are an authentication concern. Authentication is about **proving who you are**; `User` is about **who exists**. Keeping them separate means Authentication can depend on `UsersService` without owning `User`'s own data.

UsersService — the three operations User actually needs right now

TS

```ts
@Injectable()
export class UsersService {
  constructor(
    @InjectModel(User.name) private readonly model: Model<UserDocument>,
  ) {}

  findByEmail(email: string): Promise<UserDocument | null> { ... }
  async create(data: { name: string; email: string; passwordHash: string }) { ... }
  toSafeUser(user: UserDocument): SafeUser { ... }
}
```

`AuthService` never injects the `User` Mongoose model directly — it only ever calls these three methods. That's the real point of separating the two modules: Authentication USES `User`, it does not OWN it.

What is `SafeUser`?

TS

```ts
export interface SafeUser {
  id: number;
  name: string;
  email: string;
  createdAt: Date;
}
```

Notice what's missing: `passwordHash`. `toSafeUser()` builds its return value from this exact, explicit list of fields — an **allow-list**. That is the opposite of returning the raw database document and hoping nothing sensitive happens to be on it. Even if the `User` schema grows new fields later, `toSafeUser()` will never accidentally start returning them, because it only ever returns what's explicitly listed here.

Why passwords are hashed, and why bcrypt specifically

A password must never be recoverable by anyone — not an attacker who steals the database, not even this application itself. Hashing is a **one-way** transformation: there is no `bcrypt.decrypt()`, on purpose. Verifying a password never needs to recover the original value — it only ever needs to answer "does this input produce the same hash?"

`bcrypt` specifically (rather than a general-purpose hash like SHA-256) is used because:

- it generates and stores a random **salt** as part of every hash it produces, so the identical password typed by two different users (or the same user twice) never produces the identical stored hash;
- it is deliberately **slow** — its own "cost factor" (also called **salt rounds**) controls how many times the underlying algorithm repeats. This course uses `10` rounds. A slower hash makes large-scale offline password guessing against a stolen database expensive, without making verifying one real login attempt noticeably slow for a real user.

Registration, end to end

TS

```ts
async register(dto: RegisterDto): Promise<SafeUser> {
  const existing = await this.usersService.findByEmail(dto.email);

  if (existing) {
    throw new ConflictException(`An account with email "${dto.email}" already exists.`);
  }

  const passwordHash = await bcrypt.hash(dto.password, BCRYPT_SALT_ROUNDS);

  const user = await this.usersService.create({
    name: dto.name,
    email: dto.email,
    passwordHash,
  });

  return this.usersService.toSafeUser(user);
}
```

The shape should feel familiar: a uniqueness pre-check before insert is the identical pattern already used for Project `key` and Workspace `slug`, now protecting an email instead. `RegisterDto` validates the input first (`@IsEmail()`, `@MinLength(8)` on the password) — nothing reaches this method at all with a malformed email or a too-short password.

Why must `passwordHash` never be returned by the API? Every response is, in practice, data that can leave this server's own trust boundary — logs, browser devtools, a misconfigured client, a future bug. A strong hash is still a genuine secret worth protecting; the only truly safe amount of exposure for it is none. `toSafeUser()` is what guarantees this.

`POST /auth/register` — real behavior

Input	Result
Valid name/email/password	`201 Created`, a safe user (no `passwordHash`)
An email that already has an account	`409 Conflict`
An invalid email or a too-short password	`400 Bad Request`

A real mistake, found in this exact code — and the rule it teaches

One real line, written during class, writes the computed password hash straight to the server log, immediately after hashing it. **That line is wrong, and it is a bug, not a pattern to learn from.**

The rule it violates: **sensitive authentication data — a password, or a password hash — must never be logged, under any circumstance.** A hash is still a genuine secret; even though it can't be reversed back to the original password, logging it still hands an attacker who gets access to your logs a head start on an

offline guessing attack. The correct behavior is to log only non-sensitive facts about what happened (for example: "registered user 4" — an id and an email, never the hash).

Removing this exact line is the first, smallest task in your Completion Homework.

Where we stopped

We can now **register** users safely: a real account, a real one-way hash, a response that can never leak it.

We do **not** yet have a way for an already-registered user to prove they know their password. There is no `POST /auth/login`, nothing that compares a submitted password against a stored hash, and nothing that issues a token proving who someone is.

That is your Completion Homework: build login — find the account, verify the password the correct way, issue a signed token — and finish the identity-normalization improvement so an email's casing can never silently create two different accounts for the same person.

Key concepts

- Module ownership: a domain owns its own data; other modules depend on its service, never its model directly.
- Allow-list response shaping (`toSafeUser()`).
- One-way hashing vs. recoverable encryption.
- Salt and salt rounds (cost factor).
- Why sensitive authentication data must never be logged.

Important files

- `api/src/issues/dto/find-issues-query.dto.ts`, `api/src/issues/issues.service.ts` — filtering.
- `api/src/issues/schemas/issue.schema.ts` — the real compound index.
- `api/src/users/` — `User`'s own schema, service, module.
- `api/src/auth/` — registration DTO, service, controller, module.

Common mistakes

- Assuming a compound index helps any query shape that touches any of its own fields, without actually measuring — the Completion Homework STRETCH task asks you to check this directly.
- Reading `User`'s schema/data directly from `AuthService` instead of through `UsersService`.
- Returning the raw database document from an endpoint instead of an explicit allow-list.
- Logging anything that resembles a password, a hash, or (once you build it) a token.
- Calling bcrypt "encryption" — it's one-way by design, on purpose.

What to remember

- A domain (like `User`) should own its own persistence; other code should depend on its service.

- Hashing and encryption solve different problems — know which one a password needs.
- A response shape should be built from an explicit allow-list, not trimmed down from everything.
- Sensitive authentication data has no safe amount of exposure in a log.